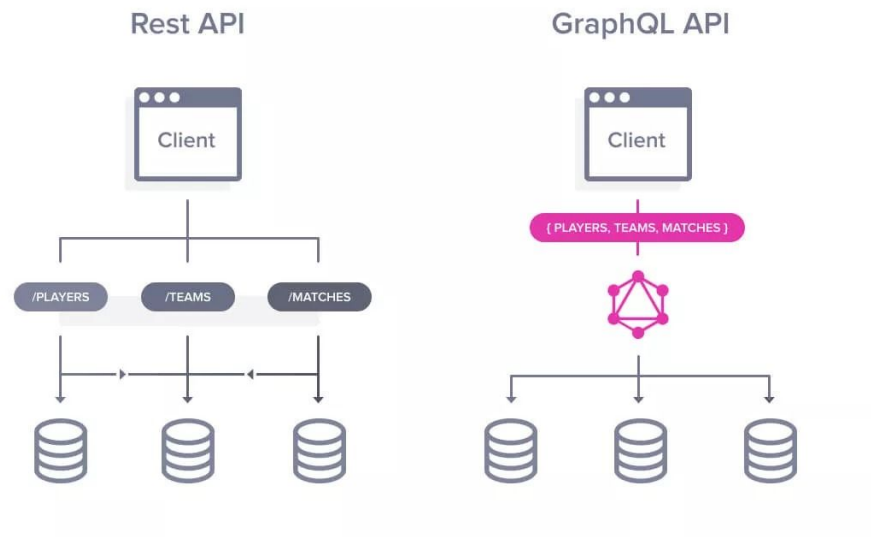


Cliente: REST + GraphQL

Implementación de consumo de servicios web utilizando libcurl y arquitectura de datos optimizada en PHP.



Carlos Daniel Kevin Apaza Villca

USFX - 2026

Concepto Híbrido

El proyecto se estructuró con una separación de responsabilidades para aislar la lógica de red de la lógica de presentación.

- ✓ **Motor Híbrido:** Centraliza las peticiones tanto para el Login (REST) como para CRUD (GraphQL).
- ✓ **Capa de Presentación:** Interfaz en PHP que procesa formularios y visualiza datos dinámicos.
- ✓ **Gestor de Sesión:** Encargado de almacenar y recuperar el JWT devuelto por la API.

```
34 /** 1. LOGIN (API REST Se comunica con la ruta tradicional de Laravel para obtener el JWT. */
35 function login(string $email, string $password): array
36 {
37     return apiRequest('http://127.0.0.1:8000/api/login', [
38         'email' => $email,
39         'password' => $password
40     ]);
41 }
42
43 /** 2. LISTAR PERSONAS (GraphQL Query) pide exactamente los campos que necesita mostrar la tabla. */
44 function getPersonas(string $token): array
45 {
46     $query = 'query {
47         personas {
48             data {
49                 id
50                 nombres
51                 apellidos
52                 documento_identidad
53                 sexo
54                 fecha_nacimiento
55                 celular
56             }
57         }
58     }';
59     return apiRequest('http://127.0.0.1:8000/graphql', ['query' => $query], $token);
60 }
61
```

Jerarquía Lógica

Cliente_Hibrido/

- |— `index.php` // Controlador de vista e interfaz
- |— `graphql-api-client.php` // Wrapper de cURL y Queries
- |— `style.css` // Estilos del Dashboard
- |— `readme.md` // Documentación técnica

Estructura del Proyecto:

La organización de archivos permite un mantenimiento ágil. Mientras que `index.php` orquesta la experiencia del usuario, el archivo `graphql-api-client.php` actúa como el motor de comunicación server-side.

libcurl en PHP

¿Qué es cURL?

Client URL Library es una biblioteca de transferencia de datos que soporta protocolos como HTTP y HTTPS, permitiendo comunicación entre servidores.

¿Por qué utilizarlo?

A diferencia de métodos simples, cURL permite un control preciso de:

- Inyección de cabeceras personalizadas.
- Gestión de seguridad (JWT Bearer).
- Envío de Payloads estructurados en JSON.

```
$curl = curl_init();  
$headers = [  
    'Accept: application/json',  
    'Content-Type: application/json'  
];  
// Si hay un token, lo inyectamos en la cabecera  
if ($token) {  
    $headers[] = 'Authorization: Bearer ' . $token;  
}
```

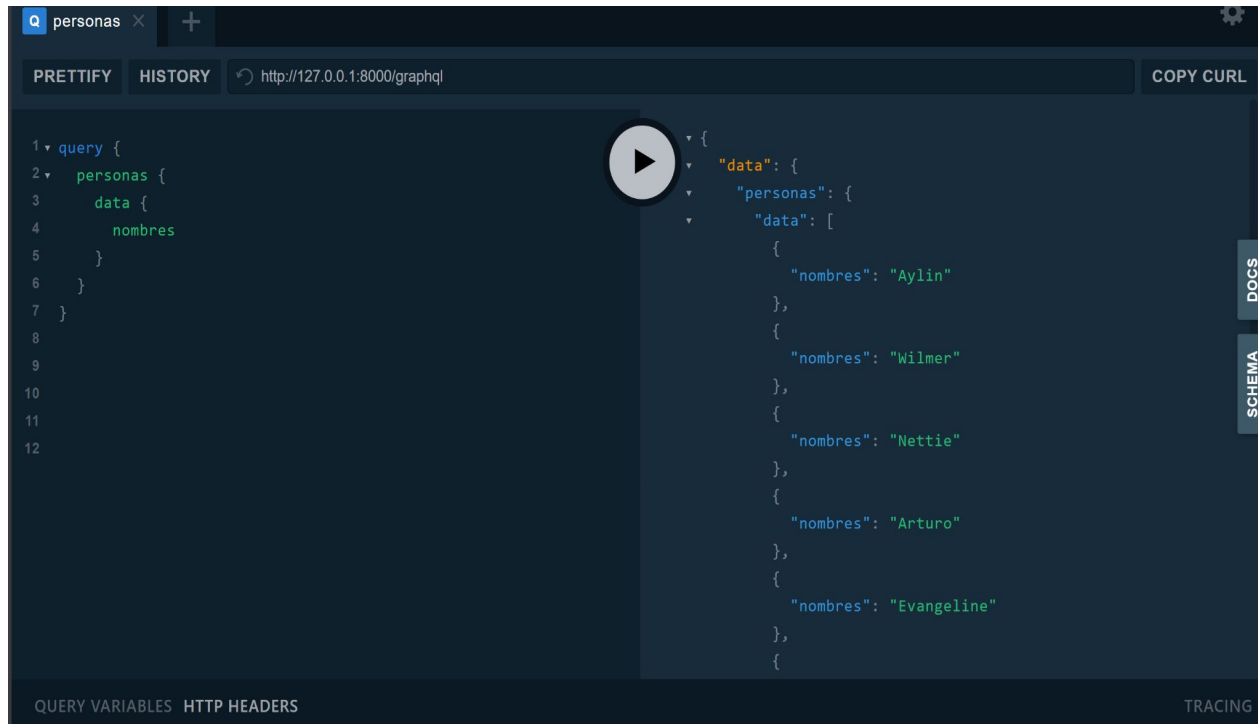
El Ciclo de Vida de cURL



Este proceso garantiza que cada petición (ya sea REST o GraphQL) se maneje de forma aislada y segura.

Eficiencia de Datos (El Problema del Overfetching)

Mientras que REST suele sufrir de **Overfetching** (envío de datos innecesarios), GraphQL permite al cliente pedir exactamente lo que necesita.



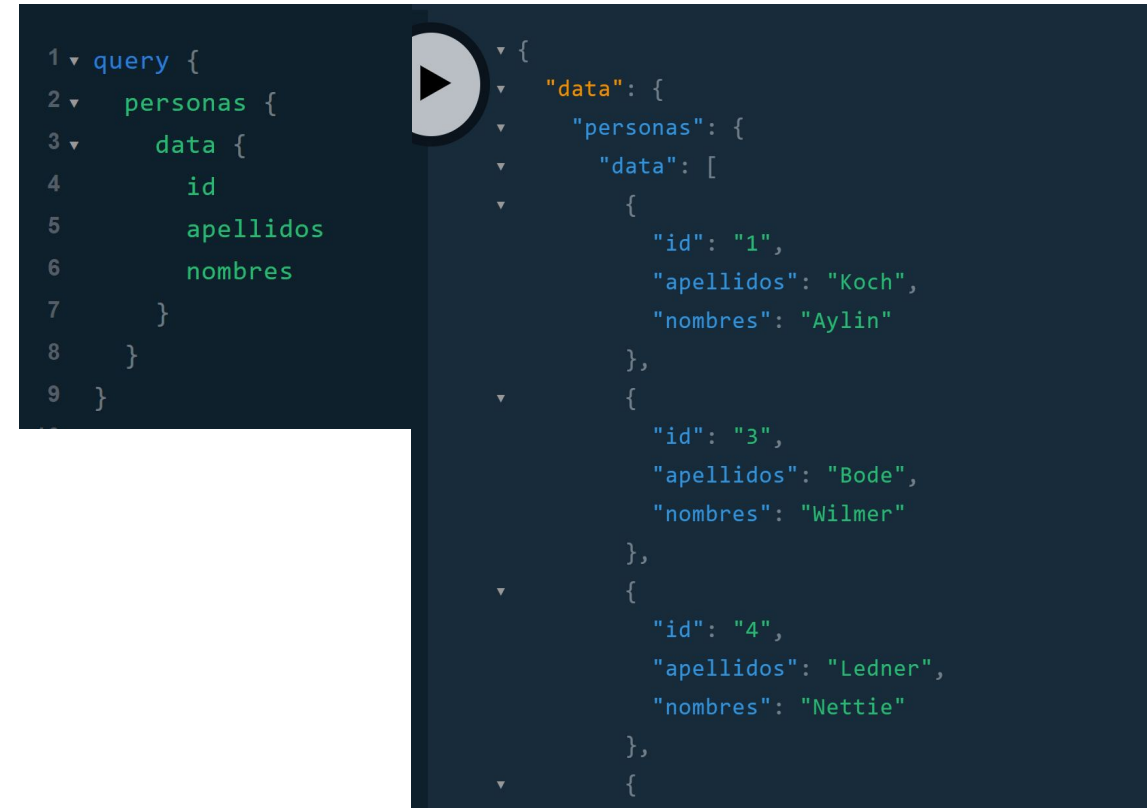
The screenshot shows the GraphQL Playground interface. On the left, a query is entered:

```
1 query {  
2   personas {  
3     data {  
4       nombres  
5     }  
6   }  
7 }  
8  
9  
10  
11  
12
```

. A play button is visible. On the right, the JSON response is displayed:

```
{  
  "data": {  
    "personas": {  
      "data": [  
        {  
          "nombres": "Aylin"  
        },  
        {  
          "nombres": "Wilmer"  
        },  
        {  
          "nombres": "Nettie"  
        },  
        {  
          "nombres": "Arturo"  
        },  
        {  
          "nombres": "Evangeline"  
        }  
      ]  
    }  
  }  
}
```

. The interface includes tabs for PRETTIFY, HISTORY, and a URL bar showing `http://127.0.0.1:8000/graphql`. On the right side, there are tabs for DOCS and SCHEMA, and a TRACING button at the bottom right.



This block compares a REST query with a GraphQL query. On the left, a REST query is shown:

```
1 query {  
2   personas {  
3     data {  
4       id  
5       apellidos  
6       nombres  
7     }  
8   }  
9 }
```

. On the right, a GraphQL query is shown:

```
{  
  "data": {  
    "personas": {  
      "data": [  
        {  
          "id": "1",  
          "apellidos": "Koch",  
          "nombres": "Aylin"  
        },  
        {  
          "id": "3",  
          "apellidos": "Bode",  
          "nombres": "Wilmer"  
        },  
        {  
          "id": "4",  
          "apellidos": "Ledner",  
          "nombres": "Nettie"  
        }  
      ]  
    }  
  }  
}
```

. A play button is visible between the two queries.

"El cliente tiene el control total sobre la respuesta del servidor."

El Motor de Comunicación (El Wrapper cURL)

Creamos una función centralizada `apiRequest()` para unificar la lógica de comunicación:

```
graphql-api-client.php > PHP Intelephense > apiRequest
1  <?php
2  function apiRequest(string $url, array $data = null, string $token = null): array
3  {
4      $curl = curl_init();
5      $headers = [
6          'Accept: application/json',
7          'Content-Type: application/json'
8      ];
9      // Si hay un token, lo inyectamos en la cabecera
10     if ($token) {
11         $headers[] = 'Authorization: Bearer ' . $token;
12     }
13
14     curl_setopt_array($curl, [
15         CURLOPT_URL => $url,
16         CURLOPT_RETURNTRANSFER => true,
17         CURLOPT_CUSTOMREQUEST => 'POST', // Siempre POST para REST-Login y GraphQL
18         CURLOPT_HTTPHEADER => $headers,
19         CURLOPT_POSTFIELDS => $data ? json_encode($data) : null,
20     ]);
21
22     $response = curl_exec($curl);
23     $error = curl_error($curl);
24     curl_close($curl);
25
26     if ($error) {
27         throw new RuntimeException('Error de cURL: ' . $error);
28     }
29
30     $result = json_decode($response, true);
31     return $result ?: [];
32 }
```

Esta es la función que centraliza mi red. En `CURLOPT_CUSTOMREQUEST` siempre está en POST.

Aquí inyectó dinámicamente el Token JWT en la cabecera, convierto mi arreglo PHP a formato JSON y lo despacho. Si hay errores de conectividad, cURL los atrapa antes de que el sistema falle

Ciclo de Vida del JWT

Obtención: Se realiza un POST a /api/login (REST). El servidor valida y retorna el token.

Almacenamiento: El cliente PHP guarda el token en \$_SESSION['api_token'], protegiéndolo de ataques XSS.

Inyección: Para cada petición GraphQL subsiguiente, el token se añade a la cabecera Authorization: Bearer.

```
try {  
    // login rest  
    if ($action === 'login' && $_SERVER['REQUEST_METHOD'] === 'POST') {  
        $email = trim($_POST['email'] ?? '');  
        $password = trim($_POST['password'] ?? '');  
  
        $loginData = login($email, $password);  
        if (isset($loginData['token'])) {  
            $_SESSION['api_token'] = $loginData['token'];  
            $token = $_SESSION['api_token'];  
            $message = 'Login exitoso. Token guardado en sesión.';  
        } else {  
            throw new RuntimeException("Error del servidor: " . json_encode($loginData));  
        }  
    }  
}  
  
// Cerrar session
```

Una vez que mi wrapper hace el login por REST, atrapo el Token JWT

Serialización y Tipos de Petición

Queries vs Mutations

En GraphQL existen principalmente los:

- Queries
- Mutations

Query solo sirve para CONSULTAR datos

Mutation sirve para MODIFICAR datos

A diferencia de REST, donde cambiamos el método HTTP (PUT/DELETE), en GraphQL siempre utilizamos POST y enviamos la intención en el código de la consulta.

```
// Query (Lectura)
$query = 'query { personas { nombres } }';

// Mutation (Escritura)
$mutation = 'mutation { crearPersona(...) }';
```

Construyendo el CRUD (Queries y Mutations)

Para insertar o editar, armo un String con sintaxis GraphQL dentro de PHP y concateno las variables limpias que vienen del formulario

```
function updatePersona(string $token, array $data): array
{
    $mutation = 'mutation {
        actualizarPersona(
            id: "'. $data['id'] .'",
            nombres: "'. $data['nombres'] .'",
            apellidos: "'. $data['apellidos'] .'",
            documento_identidad: "'. $data['ci'] .'",
            sexo: ' '. $data['sexo'] .',
            fecha_nacimiento: "'. $data['fecha'] .'",
            celular: ' '. $data['celular'] .'
        ) {
            id
        }
    }';
    return apiRequest('http://127.0.0.1:8000/graphql', ['query' => $mutation], $token);
}
```

La Interfaz y Controladores (index.php)

El archivo index.php actúa como enrutador. Evalúa la variable \$action oculta en el formulario para determinar qué función ejecutar. Además, integra un pequeño script en JS.

```
// Crear Persona (GraphQL)
if ($token && $action === 'create_persona' && $_SERVER['REQUEST_METHOD'] === 'POST') {
    $created = createPersona($token, [
        'nombres' => trim($_POST['nombres']),
        'apellidos' => trim($_POST['apellidos']),
        'ci' => trim($_POST['ci']),
        'sexo' => $_POST['sexo'],
        'fecha' => $_POST['fecha'],
        'celular' => intval($_POST['celular'])
    ]);

    if (isset($created['errors'])) {
        throw new RuntimeException("Error GraphQL: " . $created['errors'][0]['message']);
    }
    $message = 'Persona registrada exitosamente.';
}
```

```
</script>
function cargarDatos(persona) {
    document.getElementById('upd_id').value = persona.id;
    document.getElementById('upd_nombres').value = persona.nombres;
    document.getElementById('upd_apellidos').value = persona.apellidos;
    document.getElementById('upd_ci').value = persona.documento_identidad;
    document.getElementById('upd_sexo').value = persona.sexo;
    document.getElementById('upd_fecha').value = persona.fecha_nacimiento;
    document.getElementById('upd_celular').value = persona.celular;

    document.getElementById('formUpdate').scrollIntoView({ behavior: 'smooth' });
}
</script>
```

Conclusión Técnica



Rendimiento (Zero Overfetching): Al delegar el CRUD a GraphQL, erradicamos la descarga de datos basura. El cliente dicta exactamente qué columnas viajarán por la red.



Seguridad Stateless (JWT): El Token nunca toca el navegador ni el DOM del usuario. Queda aislado de forma segura en la sesión del servidor PHP (`$_SESSION`), blindando el sistema contra ataques XSS.



Mantenibilidad: Un único punto de entrada (`/graphql`) y un solo método HTTP (POST) reducen drásticamente la complejidad del cliente cURL en comparación a un cliente REST tradicional.